Introduction

What is an Algorithm?

An **algorithm** is a **finite, well-defined sequence of steps** used to solve a problem or perform a task.

Let's see a more formal definition:

An algorithm is a procedure that:

1. Takes **zero or more inputs**
2. Produces **at least one output**
3. Consists of **clear, unambiguous steps**
4. **Terminates** after a finite number of steps

Example

Problem: Find the larger of two numbers.

Algorithm:

1. Read numbers A and B
2. If A > B, output A
3. Otherwise, output B
4. Stop

Types of Problems

1. Sorting
2. Searching
3. String Processing
4. Graph
5. Combinatorial
6. Geometric
7. Numerical

Uses of Algorithms

- Solve problems step by step.
- Sort and search data efficiently.
- Power software and applications.
- Process and analyze large data.

- Enable artificial intelligence systems.
- Secure data using encryption methods.
- Manage computer memory and CPU scheduling.
- Control networks and data transmission.
- Support automation and robotics.
- Improve speed and performance of programs.

Asymptotic Notations

1. Big-O Notation

- ***f(n) grows at most as fast as g(n)***
- Upper bound/ worst-case growth
- It means that after some point, f(n) will never grow faster than a constant multiple of g(n).

  Example

  If $f(n) = 3n^2 + 4n$

  Then $f(n) = O(n^2)$

  Because it does not grow faster than n^2.

2. Big-Ω (Omega) Notation

- ***f(n) grows at least as fast as g(n).***
- It gives a **lower bound** (best-case).
- After some point, f(n) will always be at least a constant multiple of g(n).

  Example

  $$3n^2 + 5n = \Omega(n^2)$$

  Because it grows at least as fast as n^2.

3. Big-Θ (Thetta) Notation

- f(n) grows exactly as fast as g(n).
- It gives a **tight bound** (both upper and lower bound).
- That means:
  - It is not faster than g(n)
  - It is not slower than g(n)

Example

$$3n^2 + 5n = \Theta(n^2)$$

Because its true growth rate is n^2

Searching Algorithms

Searching Algorithms

Linear Search

```
Linear_search(arr,val)
//input: arr as Array to search within and val as
value to search
Step 1: i <- 0, n <- arr.length
Step 2: if i > n-1 go to Step 6
Step 3: if arr[i] not = val then go to Step 4
else go to Step 5
Step 4: i <- i+1, go to Step 2
Step 5: Print “value found” go to Step 7
Step 6: Print “value not found”
Step 7: Exit
```

GIVE THIS A THOUGHT!

Why are we using Big-O notation for worst case, best case and average case?

Reason is Big-O denotes an upper bound rather than a worst case scenario. For a best case scenario also their will be an upper bound and a lower bound and Big-O denotes that upper bound.

Time complexity:
Worst case: O(n)
Best case: O(1)
Average case: O(n)

Space complexity: O(1)

Binary Search

Iterative

```
Binary_search(arr, val):
low <- 0
high <- n-1
while(low <= high) {
      mid <- low + [(high-low // 2)]
      if arr[mid] = val then return mid
      else if arr[mid] < val then low <- mid+1
      else high <- mid-1
}
```

Space complexity: O(1)

Recursion

```
Binary_search(arr, val, low, high):
if low <= high:
      mid <- low + (high-low // 2)
      if arr[mid] = val then return mid
      else if arr[mid] > val then return Binary_search(arr, val, mid+1, high)
      else return Binary_search(arr, val, low, mid-1)
else return -1
```

Space complexity: O(logn)

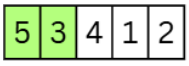
Time complexities:
Worst case: O(logn)
Best case: O(1)
Average case: O(logn)

Sorting Algorithms

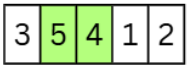
Sorting Algorithms

Bubble sort

Step 1
i=0 ; j=0; j+1=1

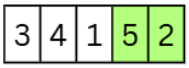


Step 2
i=0 ; j=1; j+1=2

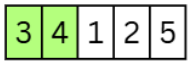


Step 3
i=0 ; j=2; j+1=3

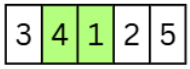
Step 4
i=0 ; j=3; j+1=4



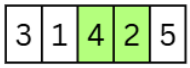
Step 5
i=1 ; j=0; j+1=1



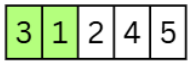
Step 6
i=1 ; j=1; j+1=2



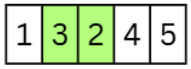
Step 7
i=1 ; j=2; j+1=3



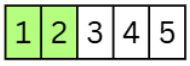
Step 8
i=2 ; j=0; j+1=1

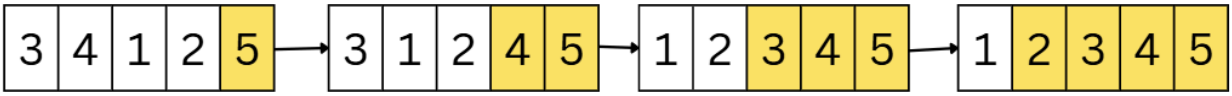


Step 9
i=2 ; j=1; j+1=2



Step 10
i=3 ; j=0; j+1=1





Algorithm

```
BubbleSort(arr){
      loop: i=0 to n-1
            loop: j=0 to n-i-2
                  if arr[j] > arr[j+1] then
                        swap arr[j] and arr[j+1]
            end loop
      end loop
}
```

Complexity

Time complexity:

- Worst Case: $O(n^2)$
- Average Case: $O(n^2)$
- Best Case: $O(n)$

Insertion Sort

```
InsertionSort(arr){
      loop: i=1 to n-1
            key = arr[i]
            j=i-1
            loop till key<arr[j] and j>=0 is true:
                  arr[j+1] = arr[j]
                  j=j-1
            arr[j+1] = key
}
```

Time Complexity:

Best Case: O(n) (already sorted array)
Average Case: $O(n^2)$
Worst Case: $O(n^2)$ (reverse sorted array)

Space Complexity: O(1) (in-place)

Merge Sort

```
mergeSort(arr){
      if len(arr) <= 1:
            return arr
      mid = len(arr)//2
      lefthalf = arr[0:mid-1]
      righthalf = arr[mid:(len(arr)-1)]

      sortedleft = mergeSort(lefthalf)
      sortedright = mergeSort(righthalf)

      return merge(sortedleft, sortedright)
}

merge(left, right){
      result = []
      i=j=0

      while i < len(left) and j < len(right){

            if left[i] < right[j]:
                  add left[i] into result
                  i = i+1
```

```
            else:
                  add right[j] into result
                  j = j+1
      }
      loop: i<len(left)
            add left[i] into result
            i = i+1
      loop: j<len(right)
            add right[j] into result
            j = j+1

      return result
}
```

Time Complexity:
Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n log n)

Space Complexity: O(n) (requires extra temporary array)

Quick Sort

```
Partition(arr, low, high){
      pivot = arr[high]

      loop: j=low to high-1{
            if arr[j] < pivot{
                  i+=1
                  swap(arr[i] and arr[j])
            }
      }

      swap(arr[i+1] and arr[high])
      return i+1
}


QuickSort(arr, low, high){
      if low<high{
            piv = partition(arr, low, high)

            quickSort(arr, low, piv-1)
```

```
            quickSort(arr, piv+1, high)
        }
}

QuickSort(arr, 0, len-1)
```

Time Complexity:

Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: $O(n^2)$ (when pivot is worst possible, e.g., already sorted with bad pivot choice)

Space Complexity:

O(log n) (due to recursion stack in average case)
Worst case: O(n)

Recursion

Recursion

What is recursion?

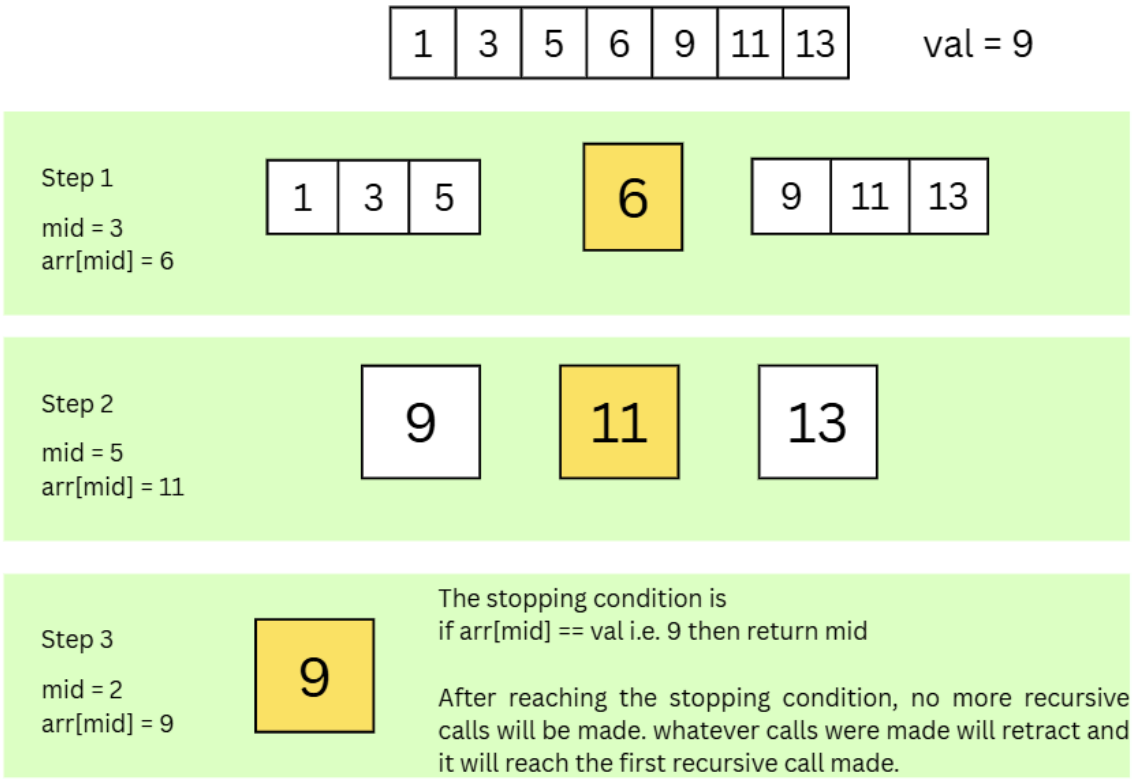
Recursion is a problem-solving technique where a function calls itself to solve smaller instances of the same problem until a **base case/ stopping condition** is reached.

A recursive algorithm has 2 main parts:

1. Base case / stopping condition
2. Recursive case

Let's take an example of a binary search recursive algorithm.

* Assuming you have already covered the binary search algorithm section.



Recursive case is the condition on which I will call my function recursively with a smaller set of input values.

If you notice, you will see that after every recursive call the input/arr becomes smaller and smaller in size. If there is no change in the input then the recursion will NEVER stop and will result in infinite recursive

calls. Along with the change in the input there is one more important thing, i.e the stopping condition. If the stopping condition is not there then the recursion will NEVER stop.

In the above condition, the recursion stopped because there is a condition which checks whether arr[mid] is equal to val or not. Else the recursion NEVER stops.

Go back and check the binary search using recursion and try to correlate the concepts.

What is a Recurrence relation?

A recurrence relation is an equation which is defined in terms of itself. A **recurrence relation** (or **recurrence**) is an equation or inequality that describes a function in terms of its value on smaller inputs.

Some of the common uses of Recurrence Relations are:

- Time Complexity Analysis
- Generalizing Divide and Conquer Algorithms
- Analyzing Recursive Algorithms

Let's discuss a very common example: Fibonacci sequence.

What is the Fibonacci sequence? 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …
What do you notice?
First 2 numbers are added to get the 3rd number and 2nd and 3rd numbers are added to get the 4th number and so on.

Question: Find the nth element in the Fibonacci sequence.
Can I write an algorithm using a normal loop? Yes.

```
def fibonacci_iterative(n):
    if n == 0:
        return 0
    if n == 1:
        return 1

    prev = 0
    curr = 1

    for i in range(2, n + 1):
        next = prev + curr
        prev = curr
        curr = next

    return curr
```

But how can I write the same thing using recursion?

We will start calculating from the nth element. At every step we are actually calculating the sum of the previous two numbers and we will stop when we get either 0 or 1.
Let's see what the code might look like.

```
def fibonacci_recursive(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)
```

Highlighted in green are the stopping conditions or the base condition which is making the recursion stop. While the yellow highlighted part is the recursion condition where we are adding the previous two numbers.

We saw the application of the recursive relations i.e. recursive functions. Now let's see how exactly a recursive relation is written.

$$F(n) = F(n-1) + F(n-2)$$

Here f(n) is defined by the function f itself. That is what recursive relation means.

Methods to solve recurrence relations:

1. Substitution method
2. Masters theorem
3. Recursion tree

Substitution Method

Steps:

1. Write the recurrence relation of the algorithm
2. Solve the recurrence relation
3. Represent the recurrence relation using asymptotic notation.

Write the recurrence relation of the algorithm

To write the recurrence relation we must know that recurrence relation is a mathematical expression that describes the overall cost of the problem in terms of the cost of solving the smaller sub problems.
Let's first take an example:

```
Algo factorial(n){
      if n==1:
            return 1
      else:
            return n * factorial(n-1)
}
```

Now when you write the recurrence relation of the above algorithm you can write the following:

$f(n) = n * f(n-1)$

As you can see the above function *f(n)* is written in terms of itself with a reduced input size wrt *n.*

Now, let's solve this using the substitution method:
T(n) -> is the time taken to execute the algorithm for a certain n>=1.
C -> is a constant time.

$$f(n) = n * f(n-1)$$

The multiplication operation takes *constant time.*
f(n) takes T(n) time and *f(n-1)* takes T(n-1) time

Thus, we can say that

$$T(n) = \begin{cases} T(n-1) + C \text{ if } n>1 \\ 1 \text{ if } n=1 \end{cases}$$

For n = 5;

$T(5) = T(4) + C$
$T(5) = T(3) + C + C$
$T(5) = T(2) + C + C + C$
$T(5) = T(1) + C + C+ C+ C$
$T(5) = 1 + C+C+C+C$
$T(5) = 1+ 4C$

Here C is the time taken to multiply any number with another number i.e. n f(n-1)*
Assuming that all the multiplication operations take the same C time.